CropLedger: Blockchain-Backed Evidence for Edge AI Crop-Field Data

Synopsis Report - 23CSE335 Blockchain

Submitted by: [Student Name] ([Roll Number]) - [Programme / Section] - October 2026

Source code: https://github.com/Roots26Hz/CropLedger

1. Project Overview and Abstract

Soil nutrient readings - nitrogen (N), phosphorus (P) and potassium (K) - from in-field sensors increasingly decide fertiliser plans, crop-insurance claims, subsidies and certification. In a dispute they are often the only objective evidence, yet they normally sit in a single vendor-controlled database where records can be edited, deleted or back-dated, and where farmers, insurers and regulators cannot independently check them.

CropLedger anchors such readings on a blockchain. A simulated edge AI device generates NPK readings together with moisture, temperature, pH and an on-device anomaly score, serialises them into a canonical payload, hashes it with keccak-256 and signs the reading with its own key using EIP-712 typed data. A gateway relays the signed reading to the CropLedger smart contract, written in Solidity for the EVM and targeted at the Polygon Amoy testnet. The contract accepts the reading only if it is signed by a registered, non-revoked device and carries a fresh nonce; it then stores compact values and the payload hash and emits an event. Any party can later retrieve a device's or field's history and verify a payload by recomputing its hash. A React dashboard integrated with MetaMask demonstrates the full workflow, including tamper detection and rejected replay, forged and unregistered-device submissions. The system protects the integrity and attribution of recorded readings; it does not, and cannot, prove that the physical sensor was accurate.

2. Problem Statement

- **Tamper-prone evidence:** centralised records can be altered after a dispute arises.
- **No verifiable origin:** nothing binds a reading to a specific registered device or approved firmware, so spoofed data is hard to distinguish from genuine data.
- **Slow, trust-based retrieval:** assembling a field's history means asking the vendor and trusting the answer.
- **No shared source of truth** between farmers, insurers, buyers and regulators.

The problem addressed is: *how can data produced by an edge device in a crop field be recorded so that it is tamper-evident, attributable to a registered device, and quickly retrievable and verifiable by any interested party?*

3. Objectives

1. Design a smart contract that stores readings immutably and exposes evidence queries.
2. Establish device identity through an owner-managed registry (field and firmware hash per device) with revocation.
3. Authenticate every reading with a device signature verified on-chain, and prevent replay.
4. Build an edge-device simulator producing realistic NPK time series, anomalies and attack attempts.
5. Make evidence easy to retrieve and verify by recomputing payload hashes.
6. Provide a dashboard showing the application, network and consensus layers, with MetaMask integration, Polygon Amoy support and a clearly labelled local demonstration mode.

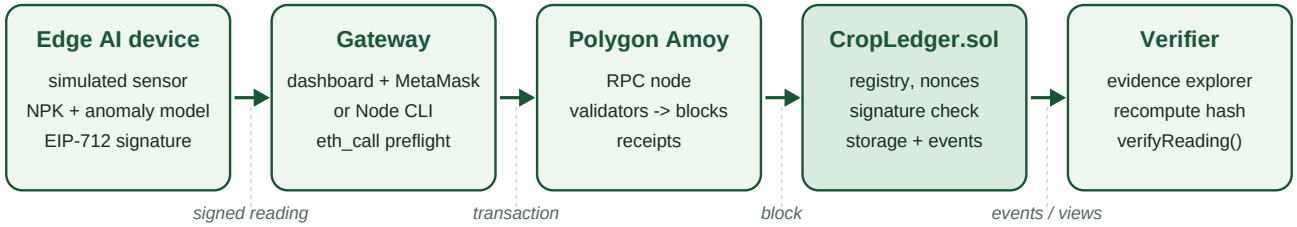
4. Proposed Methodology

The design follows the evidence chain from the field to the verifier:

1. **Canonical payload.** Every reading becomes a JSON object with a fixed key order and integer-only values (fractions are scaled, e.g. pH x 100), so the same reading always yields byte-identical text. Its keccak-256 hash is the payload hash. Unknown or missing keys are rejected.
2. **Device signature.** The device signs the EIP-712 struct *Reading(deviceId, fieldId, timestamp, n, p, k, nonce, payloadHash)* under the domain {name "CropLedger", version "1", chainId, contract address}. The domain prevents reuse of a signature on another chain or deployment; deviceId, fieldId and nonce prevent reuse across devices, fields and submissions.

3. **Untrusted gateway.** Any account may relay a reading. The gateway validates it, simulates the call (eth_call preflight, so rejections cost no gas), sends the transaction through the user's wallet, and reports *submitted* only when the RPC accepts it and *confirmed* only when a successful receipt is available.
4. **On-chain rules.** The contract checks, in order: device registered, device active, nonce greater than the last accepted one, timestamp and nutrient bounds, non-empty hash, and finally that the signature recovers to the registered signer (OpenZeppelin ECDSA).
5. **Verification.** A verifier recomputes the hash of the payload it holds and calls verifyReading(device, index, hash); any edit to the payload changes the hash and the call returns false.
6. **Separation of modes.** A real-network mode (Polygon Amoy or a local Hardhat chain) and an in-browser demonstration mode that enforces identical rules but produces no transactions and is labelled as simulated throughout.

5. System Architecture



Full payload kept off-chain; only compact values + the keccak-256 payload hash are stored on-chain.

Figure 1. CropLedger architecture and evidence flow.

Layer	Component	Role
Edge	Simulated edge AI device	Samples N, P, K, moisture, temperature, pH; rolling z-score anomaly model; canonical payload; EIP-712 signature
Network	Gateway (dashboard + MetaMask, or Node CLI)	Validation, eth_call preflight, broadcast, receipt tracking, error decoding
Consensus	Polygon Amoy validators	Order transactions into blocks (observed, not re-implemented)
Application	CropLedger smart contract	Registry, signature verification, replay protection, storage, events, evidence queries
Presentation	React dashboard	Live feed, registry, transactions, evidence explorer, verification and attack labs, network view

6. Technology Stack

Area	Tools
Smart contract	Solidity 0.8.34 (EVM cancun, optimizer 200 runs), OpenZeppelin Contracts 5.6.1 (Ownable2Step, EIP712, ECDSA)
IDE and network	Remix IDE for compiling, deploying and demonstrating the contract; Polygon Amoy testnet (chain ID 80002, POL gas)
Wallet	MetaMask via EIP-1193, with EIP-6963 wallet discovery and network switching
Development and testing	Hardhat 3.20 (Mocha + ethers), ethers.js 6.17, Vitest 5, ESLint 10, GitHub Actions
Simulator and gateway	Node.js 24 + TypeScript (shared library used by the tests, the CLI and the browser)
Dashboard	React 19, Vite 8, TypeScript 6, Tailwind CSS 4, Recharts 3

7. Implementation Details

7.1 Smart contract

CropLedger inherits OpenZeppelin's Ownable2Step (two-step ownership transfer) and EIP712. Each device record holds the signer address, last nonce, active flag, registration and revocation times, field ID and firmware hash. A device ID and a signer address can never be reused, even after revocation, so historical evidence cannot be re-attributed. Each reading is packed into two storage slots: timestamp, N, P, K, nonce and the recording block

number in one slot and the payload hash in the other. The stored block number lets clients locate the recording transaction with a single-block log query. Owner functions are registerDevice, revokeDevice and updateFirmwareHash; anyone may call submitReading; view functions include getReadingCount, getReading, verifyReading, getDevice, device enumeration and readingDigest. The contract emits DeviceRegistered, DeviceRevoked, FirmwareHashUpdated and ReadingRecorded, and reports failures through custom errors such as UnknownDevice, DeviceNotActive, StaleNonce and InvalidSignature.

7.2 Simulator, gateway and CLI

The simulator models field-specific baselines with diurnal variation, slow nutrient depletion and Gaussian noise, and can inject a spike, calibration drift or a probe dropout. Its anomaly model labels a reading as an anomaly when the largest z-score of N, P or K against a window of recent normal readings is at least 3; the label and score are part of the signed payload. The same TypeScript library supplies payload hashing, EIP-712 signing, the gateway and an in-memory copy of the contract rules, and is used by the Hardhat tests, a Node command-line tool (device keys, streaming, Remix-ready arguments, attack scenarios, verification) and the dashboard. Mined-but-reverted transactions are decoded by re-simulating the call at the previous block, because receipts carry no revert data.

7.3 Dashboard and wallet

The dashboard has eight pages: Overview, Live Sensor Feed, Device Registry, Blockchain Transactions, Evidence Explorer, Verification Lab (an eight-step integrity workflow plus replay, unregistered-device, gateway-forgery and revoked-device attacks), Network and Consensus (transaction lifecycle from real stage timestamps, blocks and confirmations, plus a validator animation labelled as illustrative), and System Configuration (mode, contract address, RPC, read-only diagnostics). MetaMask signs every write; the application never handles private keys. Only chain IDs 80002 and 31337 are accepted, explorer links point to the Amoy explorer, and VITE configuration contains no secrets. The Solidity source is shown only in Remix; the dashboard uses the ABI and the deployed address.

8. Testing and Observed Results

All results below were obtained on 18 October 2026 and are reproduced by *npm test*:

Suite	Scope	Result
Contract (Hardhat, 42 tests)	Deployment, registration, revocation, firmware, submission and retrieval, payload verification, invalid/cross-device/cross-contract/malformed signatures, replay, input validation, ownership, ABI drift	42 passed
Gateway integration (6)	Lifecycle stages, preflight rejection without broadcast, mined failure with recovered reason, wrong domain, edited payload, unsupported chain	6 passed
Local-mode parity (1 scenario, 19 checks)	Identical accept/reject outcomes for the contract and the in-browser ledger	passed
Unit tests (Vitest, 56)	Canonical payload, signing, simulator, networks, error decoding, configuration, local ledger service	56 passed

Type checking, linting, the ABI check, the production build and a clean-clone install passed, and the GitHub Actions workflow completed successfully. Measured gas on the Hardhat network: registerDevice 184,247; first submitReading of a device approx. 134,240; subsequent readings approx. 100,040. End-to-end runs against a local Hardhat node confirmed real transactions for registration and readings; a genuine payload verified true and a payload with one altered nutrient value verified false; replayed, unregistered, forged and revoked submissions were rejected with StaleNonce, UnknownDevice, InvalidSignature and DeviceNotActive respectively, while the revoked device's history remained verifiable.

Capability	Status
Contract, simulator, gateway, CLI, dashboard, local demonstration mode	Implemented
All functional and security rules; MetaMask transaction path (via an EIP-1193 shim)	Tested locally (Hardhat)
Deployment and transactions on Polygon Amoy; confirmation latency; cost in POL	Not yet performed - to be completed with the Remix deployment guide
Batching, IPFS payloads, hardware devices	Planned (future scope)

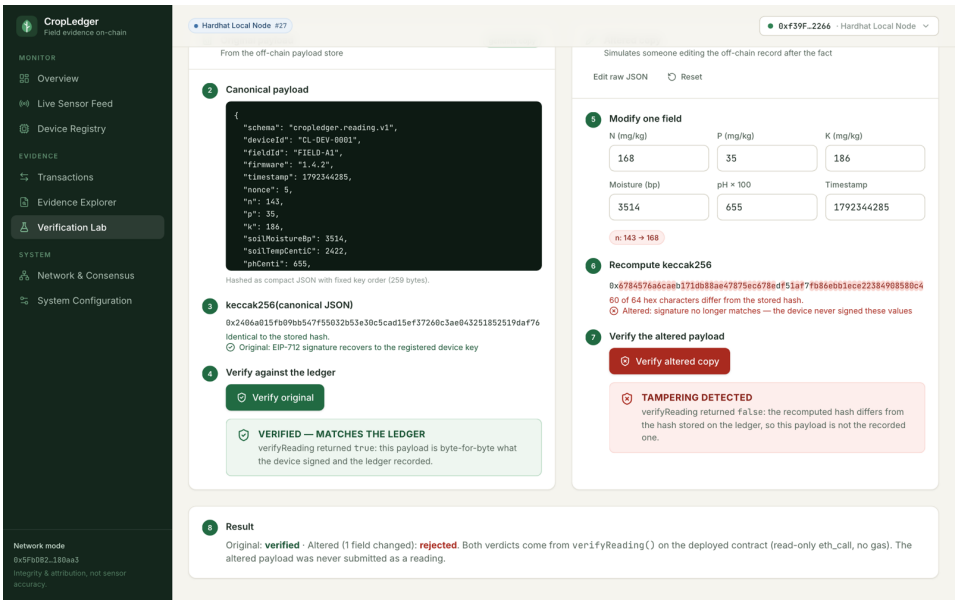


Figure 2. Verification Lab on a local Hardhat chain (real application screenshot): the stored payload verifies, while a copy with N changed from 143 to 168 is rejected by verifyReading().

9. Limitations

- The ledger proves integrity and attribution of what a device reported, not the physical accuracy of the sensor.
- Devices are simulated; their keys are held in browser storage or a local file rather than secure hardware.
- One transaction per reading is simple but expensive at scale; full payloads must be retained off-chain by their holder.
- The registry owner is a trusted single account; renouncing ownership would freeze the registry.
- No public testnet run and no automated tests of real MetaMask pop-ups have been carried out yet.

10. Future Scope

- Firmware for an ESP32 with a secure element (e.g. ATECC608) and firmware attestation.
- Merkle-batched submission of many readings per transaction, with per-reading inclusion proofs.
- Full payloads on IPFS referenced by the on-chain hash; an event indexer for large histories.
- Multisignature or consortium ownership and read-only audit roles for insurers and regulators.
- Automated insurance or subsidy logic triggered by verified readings.

11. References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
2. G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger" (Ethereum Yellow Paper).
3. R. Bloemen, L. Logvinov and J. Evans, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, 2017.
4. A. Kamilaris, A. Fonts and F. X. Prenafeta-Boldu, "The rise of blockchain technology in agriculture and food supply chains," Trends in Food Science and Technology, vol. 91, pp. 640-652, 2019.
5. OpenZeppelin, "OpenZeppelin Contracts 5.x documentation (Ownable2Step, EIP712, ECDSA)," docs.openzeppelin.com.
6. Polygon Labs, "Polygon PoS documentation: Amoy testnet and RPC endpoints," docs.polygon.technology.
7. Nomic Foundation, "Hardhat 3 documentation," hardhat.org; Remix Project, "Remix IDE documentation," remix-ide.readthedocs.io.
8. ethers.js v6 documentation, docs.ethers.org; MetaMask developer documentation (EIP-1193, wallet_addEthereumChain), docs.metamask.io.